

MODULE 03 · PART 2 - WINDOW FUNCTIONS

What a Window Actually Is

The OVER clause taken apart. Three questions, and the answer to all three has a default you did not choose.

PART 2 of 8 MODULE 3 of 26 RUNS about 30 min

A window function computes a value for a row **using other rows**, without removing any of them.

Three properties, and they are the whole family

- ◆ **One value out for every row in.** A window function never changes the number of rows. Not once, not ever.
- ◆ **The value may depend on other rows.** That is the only thing an ordinary expression cannot do.
- ◆ **Which other rows is entirely up to you,** and the OVER clause is where you say so.
- ◆ **If you did not say, the engine chose.** Every unstated part of an OVER clause has a default.

THE TAKEAWAY

The first point is the one to hold on to. If a query's row count changed, a window function was not the thing that changed it.

A window is not a loop

- ◆ **It is not evaluated once per row in sequence.** The engine sorts once and sweeps, computing every row's answer in one pass.
- ◆ **A row cannot see a value another row computed.** Windows read input columns, never each other's output.
- ◆ **Which is why a running balance that compounds is not expressible** as a single window. Each step would need the previous step's result.
- ◆ **Recursion is the tool for that**, and it is module eleven. Windows handle sequences, not feedback.

THE TAKEAWAY

'For each row, look at other rows' describes the meaning. It does not describe an iteration, and the difference decides what is expressible at all.

The same total, two shapes

GROUP BY

Rows are replaced

```
1 SELECT store_key,  
2        SUM(net_amount) AS sales  
3 FROM fact_sales_line  
4 GROUP BY store_key;  
5  
6 -- 412 rows out
```

Two billion rows in, 412 rows out. The detail is gone and cannot be referred to again.

OVER

Rows are kept

```
1 SELECT store_key, sale_key,  
2        net_amount,  
3        SUM(net_amount) OVER (  
4          PARTITION BY store_key  
5        ) AS store_sales  
6 FROM fact_sales_line;  
7  
8 -- 2.41 billion rows out
```

Every row keeps its own detail and gains its store's total alongside it.

THE TIEBREAKER

Identical arithmetic, identical totals, completely different output shape. Which one you want is decided by what the consumer needs, not by which is cleverer.

Aggregate or window

QUESTION	AGGREGATE, OR WINDOW
What does one output row represent?	A group. Or an input row, unchanged.
How many rows come out?	One per group. Or exactly as many as went in.
Can I still see the detail?	No, it was replaced. Or yes, it is right there.
Can I compare a row to its group?	Not without a second pass. Or yes, on one line.
When does it run?	Step three. Or step five, on the output of step three.

THE TAKEAWAY

The fourth row is why window functions exist. Comparing a row to the set it belongs to used to require reading the table twice.

The four families

FAMILY	MEMBERS, AND WHAT THEY NEED
Aggregate	SUM, AVG, COUNT, MIN, MAX. Frame-sensitive. Module seven.
Ranking	ROW_NUMBER, RANK, DENSE_RANK, NTILE. No frame. Module four.
Navigation	LAG, LEAD, FIRST_VALUE, LAST_VALUE, NTH_VALUE. Module six.
Distribution	PERCENT_RANK, CUME_DIST, PERCENTILE_CONT. Module nine.

THE TAKEAWAY

Four families and one OVER clause. Every function below uses the same three questions, which is why the clause is worth more attention than the function list.

Older than you think, and newer than you think

- ◆ **Standardised in SQL:2003.** Oracle shipped them in 1999, so the idea is older than most warehouses running today.
- ◆ **Postgres got them in 8.4, in 2009.** SQL Server had ranking in 2005 and frames only in 2012.
- ◆ **MySQL and SQLite both waited until 2018.** Which is why a great deal of working SQL was written without them.
- ◆ **So inherited code avoids them for reasons that expired.** A self-join written in 2015 was not necessarily a bad decision then.

THE TAKEAWAY

When you find a correlated subquery doing a window's job, check the date before judging it. The alternative may not have existed on that platform.

GROUP BY answers **what is the total**. A window answers **what is this row's relationship to the total**.

The emptiest window there is

```
1 SELECT store_key,  
2        net_amount,  
3        SUM(net_amount) OVER () AS grand_total,  
4        net_amount  
5        / SUM(net_amount) OVER () AS share  
6 FROM fact_sales_line  
7 WHERE date_key = 20260831;
```

- 3 An empty OVER means one partition containing every row, with no ordering and no frame.
- 5 The grand total is repeated on every row, which is exactly what makes the division possible.

THE TAKEAWAY

It looks wasteful to repeat one number two million times. It is the only arrangement in which a row can divide by it.

What the defaults actually are

- ◆ **No PARTITION BY.** The whole result set is one partition. Not one partition per row.
- ◆ **No ORDER BY.** No sequence, so every row sees the entire partition and they all get the same answer.
- ◆ **ORDER BY present, frame absent.** On most engines a frame appears anyway, and it is not the one people expect. Module five.
- ◆ **So OVER () is safe and predictable everywhere.** It is adding the ORDER BY that changes the meaning, and engines disagree about how.

THE TAKEAWAY

An empty OVER has no hidden behaviour at all. The moment an ORDER BY goes inside it, a frame you did not write comes with it.

PARTITION BY, which rows are candidates

```
1 SELECT s.region_key, f.store_key,  
2        SUM(f.net_amount) OVER (  
3          PARTITION BY s.region_key  
4        ) AS region_sales,  
5        SUM(f.net_amount) OVER () AS all_regions  
6 FROM fact_sales_line f  
7 JOIN dim_store s ON s.store_key = f.store_key;
```

- 3 Seven regions, so seven partitions. Each row sees only the rows sharing its region.
- 5 Two windows in one query, with different partitions. They are computed independently.

THE TAKEAWAY

PARTITION BY narrows which rows a row can see. It never removes a row, and it never merges two rows into one.

PARTITION BY and GROUP BY

GROUP BY

Collapses

- One output row per group
- Detail rows cease to exist
- Runs at step three
- Applies to the whole query

PARTITION BY

Scopes

- Output row count unchanged
- Detail rows all survive
- Runs at step five
- Applies to one function call

THE TIEBREAKER

The last row is the one people underrate. A query has one GROUP BY and may have a dozen different partitions, each scoped to its own function.

What may go inside PARTITION BY

- ◆ **Columns and expressions.** Anything computable from the row, including function calls and CASE expressions.
- ◆ **Several of them, comma-separated.** The partition is then the distinct combination, exactly as GROUP BY would be.
- ◆ **Not a window function.** Step five cannot depend on step five, which module one settled.
- ◆ **Not a bare constant in Redshift.** The documentation rules out string literals in PARTITION BY, and it is a real error, not a warning.

THE TAKEAWAY

Aliases are the awkward case. Some engines resolve a SELECT alias here and some do not, so repeating the expression is the portable choice.

Partitioning by an expression

```
1 SELECT sold_ts, store_key, net_amount,  
2       SUM(net_amount) OVER (  
3         PARTITION BY store_key,  
4         DATE_TRUNC('month', sold_ts)  
5       ) AS store_month_total  
6 FROM fact_sales_line;
```

4 A composite partition. One per store per calendar month, formed without a join to dim_date.

THE TAKEAWAY

Kestrel reports on a 4-4-5 retail calendar, so DATE_TRUNC gives the wrong month boundaries. Joining to dim_date and partitioning by retail_month_key is the correct version.

One partition column, or several

PARTITION BY A

Coarse

- Fewer, larger partitions
- Each row sees more neighbours
- One sort key
- Share of the wider group

PARTITION BY A, B

Fine

- More, smaller partitions
- Each row sees fewer neighbours
- Composite sort key
- Share of the narrower group

THE TIEBREAKER

Adding a partition column never changes the row count and always changes the numbers. It is the denominator of every share you compute.

A partition is not a filter

- ◆ **PARTITION BY region_key = 3 does not restrict to region three.** It creates two partitions, true and false.
- ◆ **And every row still comes back.** Rows outside region three are in the other partition, not excluded.
- ◆ **The misreading is understandable,** because partition sounds like a slice, and in table design it is one.
- ◆ **Filtering is WHERE, before the window, or QUALIFY, after it.** The OVER clause has no role in it at all.

THE TAKEAWAY

Partition by a boolean and you get two partitions. That is a perfectly legal and occasionally useful thing to do, and it is never what somebody writing it meant.

An analyst builds a store summary for August. The query groups by `store_key` and sums `net_amount`. To add context they include `COUNT(*) OVER ()` in the select list, expecting the total number of transactions in the month.

The figure comes back as 412 on every row. The commentary published with the report reads 'Kestrel recorded 412 transactions in August'. Nobody queries it for three days.

Why is the figure 412?

- A `COUNT(*) OVER ()` counts distinct values, and there are 412 distinct stores in the data.
- B The window needs `PARTITION BY store_key` to see the sales lines belonging to each store.
- C `COUNT(*)` should be `COUNT(sale_key)`, because the star form only counts rows that are fully populated.
- D The window runs after `GROUP BY`, so it counts the 412 grouped rows rather than the sales lines.

◆ PAUSE AND DECIDE

Why is the figure 412?

- A** It counts distinct values
COUNT() has no distinct behaviour at all. The figure matches the store count for a different reason, which is what makes this tempting.*
- B** Add PARTITION BY store_key
Gives 1 on every row, since each grouped row is alone in its own partition. It makes the answer worse and looks like a fix.
- C** Use COUNT(sale_key)
Invents a rule. COUNT() counts rows regardless of nulls, and sale_key is not null anyway. The input is the problem, not the function.*
- D** The window runs after grouping
Step three produced 412 rows and step five counted them. The sales lines stopped existing two steps earlier.

THE SIGNAL

"412 on every row." A window figure that exactly matches the output row count is counting the output, which is nearly always the grouped rows rather than the data.

The ORDER BY inside OVER is **not** the ORDER BY of your query. They are unrelated clauses.

Two ORDER BY clauses, one query

INSIDE OVER

Sequence for the function

```
1 ROW_NUMBER() OVER (  
2   PARTITION BY store_key  
3   ORDER BY sold_ts, sale_key  
4 ) AS seq
```

Decides which line is first within a store. Says nothing about how rows come back.

THE QUERY'S OWN

Sequence for the output

```
1 SELECT ...  
2   FROM fact_sales_line  
3   ORDER BY store_key,  
4             net_amount DESC
```

Decides the order rows are returned. Says nothing about how any window was computed.

THE TIEBREAKER

Snowflake's own documentation makes the point: the ORDER BY inside OVER controls order within the window, not the order of the query's output.

What ORDER BY inside OVER is for

- ◆ **Ranking needs it.** ROW_NUMBER, RANK and NTILE are meaningless without a sequence to rank along.
- ◆ **Navigation needs it.** Previous and next only mean something once the rows are in an order.
- ◆ **Aggregates do not need it, and change behaviour when they get it.** Adding it turns a total into a running total.
- ◆ **Module two applies here in full.** If the ordering is not unique within the partition, the answer is not unique either.

THE TAKEAWAY

Adding an ORDER BY to an aggregate window is not a formatting change. It converts SUM over the partition into SUM up to this row, silently.

One clause added, two different numbers

```
1 SELECT sold_ts, net_amount,  
2        SUM(net_amount) OVER (  
3          PARTITION BY store_key  
4        ) AS store_total,  
5        SUM(net_amount) OVER (  
6          PARTITION BY store_key  
7          ORDER BY sold_ts, sale_key  
8        ) AS running_total  
9 FROM fact_sales_line;
```

7 The only difference between the two windows. One line, and the meaning changes completely.

THE TAKEAWAY

Same function, same partition, one extra line. The first is the store's total on every row. The second climbs from zero to that total.

The three questions an OVER clause answers

CLAUSE	THE QUESTION, AND THE DEFAULT
PARTITION BY	Which rows are candidates? Default: all of them, as one partition.
ORDER BY	In what sequence? Default: none, so the partition is unordered.
frame	Which candidates count for this row? Default: depends on the ORDER BY.
OVER ()	All rows, no sequence, whole partition. Fully predictable.
OVER (ORDER BY x)	All rows, sequenced by x, and a frame you did not write.

THE TAKEAWAY

Three questions and three defaults. Two of the defaults are harmless. The third is module five, and it is the reason that module is the longest in the course.

Ranking functions ignore the frame

- ◆ **ROW_NUMBER, RANK, DENSE_RANK, NTILE, PERCENT_RANK, CUME_DIST.** None of them accepts a frame clause.
- ◆ **They always see the whole partition,** in the order you gave, which is why they are simpler to reason about.
- ◆ **Writing a frame with one of them is an error** in most engines, and silently ignored in the rest.
- ◆ **Aggregates, FIRST_VALUE, LAST_VALUE and NTH_VALUE do accept frames.** That split is worth memorising now.

THE TAKEAWAY

One family is frame-sensitive and one is not. Most of what people call window function weirdness lives entirely in the first family.

The window, and what it replaced

BEFORE WINDOWS

A correlated subquery

- Read the table once per row
- Or join to an aggregated copy
- Grain errors from the join
- Quadratic on a large table

WITH WINDOWS

One pass, one sort

- Read the table once
- No join, so no fan-out
- Grain cannot change
- Sort, then a single scan

THE TIEBREAKER

The performance argument is real and it is not the main one. The correctness argument is: a window cannot change your grain, and a join can.

A nightly job computes a running ledger balance per store using a correlated subquery, summing every earlier posting for each row. It has run for four years and now takes six hours, up from forty minutes.

The ledger holds 148 million rows and grows by about 120,000 a night. The team proposes doubling the cluster size. Nothing about the calculation has changed and the output is correct.

What should be done first?

- ☐ A Double the cluster. The data has grown and the compute has not, so this is a capacity problem.
- ☐ B Materialise the balance incrementally, storing yesterday's closing figure and adding today's postings.
- ☐ C Rewrite it as SUM over an ordered window, which computes the same figure in one pass instead of one pass per row.
- ☐ D Partition the ledger table by store so that each subquery scans less data.

◆ PAUSE AND DECIDE

What should be done first?

A Double the cluster

Buys back roughly half the runtime against a cost that grows with the square of the rows. Next year it is six hours again.

C Rewrite as an ordered window

Removes the quadratic term rather than paying for it. One sort and one scan, same arithmetic, and it is a single query change.

B Materialise incrementally

The right long-term design and a bigger change. It needs restatement handling, because ledger postings can be back-dated 45 days.

D Partition by store

Helps, and leaves the shape intact. Each store's subquery is still quadratic in that store's own rows, which is where the growth is.

THE SIGNAL

"Forty minutes to six hours" while rows roughly doubled. A nine-fold increase from a two-fold input is the signature of quadratic work, and no amount of hardware changes the exponent.

A window function can tell you anything about a row. It cannot **remove** one.

Four things a window cannot do

- ◆ **It cannot filter.** Filtering happens at step two, or after step six. Never at step five.
- ◆ **It cannot change the grain.** One row in, one row out, so the grain you had is the grain you keep.
- ◆ **It cannot nest inside another window function.** One step means one pass. Stack CTEs instead.
- ◆ **It cannot appear in WHERE, GROUP BY or HAVING**, all of which are evaluated before it exists.

THE TAKEAWAY

All four are the same fact restated. A window function is one step in a fixed order, and a step cannot reach backwards or act on itself.

Where a window function may appear

CLAUSE	PERMITTED?
SELECT list	Yes. This is where nearly all of them live.
ORDER BY	Yes. It runs at step nine, after the window at step five.
QUALIFY	Yes. That is the entire purpose of the clause.
WHERE, GROUP BY, HAVING	No. All three run before step five.
Inside an aggregate	No. SUM of a window function is not a legal expression.
Inside another window	No. Use two query blocks.

THE TAKEAWAY

Three yeses and three noes, and you can derive every one of them from the step number. Nothing here needs memorising separately.

A window used only for sorting

```
1 SELECT store_key, product_key, net_amount
2   FROM fact_sales_line
3  WHERE date_key = 20260831
4  ORDER BY SUM(net_amount) OVER (
5           PARTITION BY store_key
6           ) DESC,
7         net_amount DESC,
8         sale_key;
```

- 4 The window never appears in the select list. ORDER BY runs at step nine, so it can still see it.
- 8 And module two still applies. The last column makes the ordering total.

THE TAKEAWAY

Busiest stores first, then biggest lines within each. Two sort keys and no extra column in the output, because ORDER BY does not require you to project what you sort by.

Partitioning on a nullable column

- ◆ **PARTITION BY collects**, so all nulls form one partition. Module two's table, now with consequences.
- ◆ **62% of Kestrel's sales lines have no promo_key**. Partitioning by it creates one enormous partition of 1.5 billion rows.
- ◆ **That partition is not a promotion**. It is every line that was not on one, treated as though it were a single campaign.
- ◆ **Filter or split before partitioning**, so that the absent group is excluded deliberately rather than aggregated by accident.

THE TAKEAWAY

The output is not wrong in any way the engine could detect. One of the partitions simply means something the report does not say it means.

The promotion that was never run

```
1 SELECT promo_key,  
2        AVG(net_amount) OVER (  
3          PARTITION BY promo_key  
4        ) AS avg_line_value,  
5        COUNT(*) OVER (  
6          PARTITION BY promo_key  
7        ) AS lines_in_promo  
8 FROM fact_sales_line;
```

3 6,840 real promotions, plus one partition of nulls holding more rows than all of them combined.

THE TAKEAWAY

Sorted by lines_in_promo descending, the top row of this report is a promotion with no key, no name and no dates. It will be read as the biggest campaign of the year.

A promotions analyst reports average line value by campaign, using AVG over a window partitioned by promo_key across all of August. The report is sorted by line count descending and the top twenty are reviewed weekly.

The top row shows 47 million lines at an average of \$4.18, with a blank campaign name. It has appeared every week since the report was built, and the team has started referring to it as the always-on promotion.

What is the top row?

- ☐ A A genuine always-on mechanic whose campaign name was never populated in dim_promotion.
- ☐ B Every line with no promotion at all, collected into a single null partition.
- ☐ C A join defect, where lines failed to match dim_promotion and fell through to a default row.
- ☐ D Duplicate sales lines from a reload, which would inflate one partition without affecting the others.

◆ PAUSE AND DECIDE

What is the top row?

- A

 An unnamed real campaign
The reading the team arrived at, and it is why the row survived three months of weekly review. A real campaign would have a key.
- B

 The null partition
PARTITION BY treats all nulls as one group, and 62% of lines carry no promotion. The report shows a campaign that does not exist.
- C

 A failed join to a default row
Would produce the same shape and is worth ruling out. A default row has a key, so it would carry a name rather than a blank.
- D

 Duplicates from a reload
Duplicates inflate a count and leave the average roughly unchanged. They also would not settle at the same figure every week.

THE SIGNAL

"Every week since the report was built." A defect that is perfectly stable is usually not a defect in the data. It is the query describing something real and labelling it wrongly.

Several windows, one query

```
1 SELECT store_key, product_key, net_amount,  
2        SUM(net_amount) OVER (PARTITION BY store_key) AS store_total,  
3        SUM(net_amount) OVER (PARTITION BY product_key) AS product_total,  
4        AVG(net_amount) OVER () AS overall_avg,  
5        RANK() OVER (PARTITION BY store_key  
6                     ORDER BY net_amount DESC) AS rank_in_store  
7 FROM fact_sales_line  
8 WHERE date_key = 20260831;
```

2 Four windows, three distinct specifications. Each is computed independently of the others.

THE TAKEAWAY

Perfectly ordinary and perfectly legal. Each distinct specification generally costs a sort, so three specifications is three sorts unless the engine can share one.

A window sees the joined result

- ◆ **Step five runs after step one.** So the rows a window sees are the rows the join produced, not the rows the table holds.
- ◆ **If the join fanned out, the window sums the duplicates** as faithfully as it sums anything else. It has no way to know.
- ◆ **Aggregate first, then join, when the join can fan out.** Or join to something proven unique on the join key.
- ◆ **The window is not the defect here.** It is reporting a grain that was already wrong when it arrived.

THE TAKEAWAY

A window cannot change your grain, which is usually a safety property. When the grain is already wrong, the same property means it cannot rescue you either.

Fan-out, and the window that repeats it

```
1 SELECT f.sale_key, f.net_amount,  
2        SUM(f.net_amount) OVER (  
3          PARTITION BY b.category_key  
4        ) AS category_total  
5 FROM fact_sales_line f  
6 JOIN bridge_product_category b  
7   ON b.product_key = f.product_key;
```

7 A product in three categories produces three rows. The window then counts its value three times.

THE TAKEAWAY

The category totals will sum to more than the business took. Nothing in the query is invalid, and the only clue is that a total exceeds a total you trust.

Partitions of one, and partitions of none

- ◆ **A partition with one row is legal and common.** The window still runs, and the row is its own entire context.
- ◆ **So share of total is 100%, and rank is 1,** for every store that traded once. Correct, and easy to misread as a top performer.
- ◆ **LAG and LEAD return null there,** because there is no previous row. That null then flows into any arithmetic.
- ◆ **There is no such thing as an empty partition.** Partitions are formed from rows that exist, so a store with no sales has none.

THE TAKEAWAY

The last point catches reports out. A window can never produce a row for a store that sold nothing, because there was no row to compute one on.

What a window costs at runtime

- ◆ **Partition, then sort, then one sweep.** That is the shape of it in every engine, distributed or not.
- ◆ **PARTITION BY decides data movement.** Rows of one partition must meet, so a new partition key means a redistribution.
- ◆ **One enormous partition is the worst case.** All the work lands on one worker, and OVER () is exactly that shape.
- ◆ **Similar specifications are cheaper than different ones,** because an engine can often reuse one sort for several windows.

THE TAKEAWAY

OVER () is the simplest clause to write and the least parallel thing in the language. On a small result that is free, and on a billion rows it is not.

Reading an OVER clause out loud

- ◆ 'For each row, look at ...' then say the partition. If there is none, say 'every row in the result'.
- ◆ '... put them in order of ...' then say the ordering. If there is none, say 'in no order'.
- ◆ '... take ...' then say the frame. If there is none, say 'all of them' or 'everything up to here'.
- ◆ '... and compute ...' then the function. If the sentence is strange out loud, the query is strange.

THE TAKEAWAY

Every ambiguity in this part of the course survives in writing and dies when spoken. Four phrases, in that order, and you have read the clause correctly.

Writing a window function, four decisions

01

Shape

Do you need one row per group, or every row kept? That picks GROUP BY or OVER.

02

Scope

Which rows should this row be compared against? That is PARTITION BY.

03

Sequence

Does order matter for this function? Only add ORDER BY if it does.

04

Extent

If you added an order, decide the frame rather than inheriting one.

THE TAKEAWAY

Step three is where most errors are made, because people add an ordering out of habit and inherit a frame with it. If the function does not need a sequence, do not give it one.

Kestrel publishes a daily store summary table, one row per store, consumed by finance and by three dashboards. An engineer proposes replacing it with a view over the sales fact that uses window functions, arguing that windows are faster than GROUP BY and avoid a scheduled job.

The summary carries nine measures. The consumers join to it on store_key and expect exactly one row per store per day. The fact table holds 2.41 billion rows.

What should the reviewer say?

- A** Reject it. The consumers need one row per store, and a window returns one row per input row.
- B** Approve it with a DISTINCT in the view, which restores one row per store without needing the job.
- C** Approve it. Window functions read the table once, so the view will outperform the scheduled aggregate.
- D** Reject it, because a view cannot be indexed and finance requires the table to be materialised.

◆ PAUSE AND DECIDE

What should the reviewer say?

- A

Reject on shape
The consumers' contract is one row per store. A window cannot produce that, because it returns one row per input row by definition.
- B

Approve with a DISTINCT
The most dangerous option. It does produce one row per store, after materialising billions of rows and sorting them to throw almost all away.
- C

Approve, windows are faster
Compares the wrong things. GROUP BY also reads the table once, and returns 412 rows where the window returns 2.41 billion.
- D

Reject on indexing
Lands on the right verdict through an argument that is not the reason. It would also be answered by materialising the view, and the shape would still be wrong.

THE SIGNAL

"Exactly one row per store per day." That is a grain requirement, and grain is decided before performance. A window function cannot satisfy it at any speed.

Detail and context, in one result

```
1 SELECT s.region_key, f.store_key, f.net_amount,  
2        f.net_amount / SUM(f.net_amount)  
3        OVER (PARTITION BY s.region_key)      AS share_of_region,  
4        f.net_amount - AVG(f.net_amount)  
5        OVER (PARTITION BY s.region_key)      AS vs_region_avg,  
6        RANK() OVER (PARTITION BY s.region_key  
7                   ORDER BY f.net_amount DESC) AS rank_in_region  
8 FROM fact_sales_line f  
9 JOIN dim_store s ON s.store_key = f.store_key
```

3 Share, distance from the mean, and rank. Three relationships between a row and its region, on one pass.

THE TAKEAWAY

None of these is expressible without windows, and all three share one partition. Writing them with a matching PARTITION BY is what lets the engine sort once.

The five mistakes people make first

- ◆ **Expecting the row count to change.** It will not, and DISTINCT is not the fix.
- ◆ **Adding ORDER BY to an aggregate window by habit.** It turns a total into a running total, silently.
- ◆ **Partitioning on a nullable column** and getting one partition that is not a category.
- ◆ **Putting a window over a fanned-out join** and reporting a grain that was already wrong.
- ◆ **Trying to filter on the result in WHERE**, which is module one and will happen anyway.

THE TAKEAWAY

Four of these produce a number. One produces an error. The error is the only one you can rely on finding.

The vocabulary, settled

TERM	WHAT IT MEANS PRECISELY
Window function	A function computing one value per row, using other rows.
Partition	The set of rows a given row is allowed to see.
Frame	The subset of that partition in scope for this row. Module five.
Window specification	A partition, an ordering and a frame, taken together.
Ranking family	Takes no frame. Always sees the whole partition.
Aggregate family	Takes a frame. This is where the surprises live.

THE TAKEAWAY

Partition and frame get used interchangeably and they are not the same. The partition is what you could see. The frame is what you do see.

You now have two of the three questions.
Module four ranks, and module five
answers the third.